UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE
Semester Genap 2025/2026

MODUL

Nama Mahasiswa	:	Saprina Melita Sari
NIM	:	23051430012
Kelas	:	A
Tanggal Praktikum	:	27 Maret 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	MODUL 6 (Pertemuan 6)
Topik Utama	DOM Manipulation & Event Handling
Sub-Topik	Document Object Model (DOM), Seleksi Elemen, dan Event Listeners.
Durasi Praktikum	340 Menit
Tanggal Praktikum	27 Maret 2026
Nama Mahasiswa	Saprina Melita Sari
NIM	23051430012
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami struktur DOM dan bagaimana JavaScript melihat halaman web.
2	Mampu memanipulasi elemen HTML (mengubah teks, warna, kelas) secara dinamis menggunakan JavaScript.
3	Mampu menangani interaksi pengguna (klik tombol, ketik keyboard) menggunakan Event Listeners.
4	Menerapkan konsep ini untuk membuat simulasi Control Panel mesin industri.

C. ALAT DAN BAHAN

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
5	HTML	-	Struktur

6	CSS		Styling

D. DASAR TEORI

DOM Manipulation dan Event Handling dalam pengembangan web didasari oleh Document Object Model (DOM), yang merupakan antarmuka pemrograman untuk merepresentasikan dan memanipulasi dokumen HTML sebagai pohon node (Galahartlambang et al., 2021). DOM memungkinkan JavaScript berinteraksi secara dinamis dengan elemen halaman, mengubah struktur, konten, dan gaya tanpa memuat ulang halaman, sehingga mendukung aplikasi web interaktif modern. Di Indonesia, penerapan DOM sering dibahas dalam konteks performa aplikasi web untuk mengatasi keterlambatan rendering pada elemen besar.

Seleksi elemen DOM menggunakan metode seperti getElementById(), querySelector(), dan querySelectorAll() yang memanfaatkan selector CSS untuk akses presisi (Rosidi, 2025). Teknik ini esensial untuk manipulasi targeted, di mana hasil seleksi berupa objek node siap untuk operasi seperti insert atau delete, meningkatkan efisiensi kode JavaScript. Studi lokal menekankan seleksi ini dalam pengembangan sistem berbasis web untuk mengoptimalkan navigasi pohon DOM.

Event Handling direalisasikan melalui addEventListener() yang meregistrasikan fungsi callback pada event seperti 'click' atau 'input' (Galahartlambang et al., 2021). Fitur ini mendukung event delegation dan pencegahan propagasi, memungkinkan respons real-time terhadap interaksi user tanpa konflik multiple handler. Di konteks Indonesia, event handling dikaitkan dengan peningkatan user experience pada aplikasi single-page.

Teori keseluruhan menyoroti evolusi DOM dari abstraksi statis ke Virtual DOM untuk performa lebih baik, dengan aplikasi di framework seperti React dan Vue (Galahartlambang et al., 2021). Standarisasi W3C diadaptasi secara luas di penelitian lokal untuk mengukur metrik seperti durasi eksekusi dan penggunaan memori, mendukung pengembangan web skalabel.

D.1 Konsep Utama

Document Object Model (DOM) merupakan representasi struktural dari dokumen HTML dalam bentuk pohon node yang dapat dimanipulasi secara dinamis oleh JavaScript. Browser mem-parsing kode HTML menjadi objek DOM, sehingga memungkinkan pengaksesan elemen melalui metode seleksi seperti querySelector(), modifikasi konten dengan innerHTML, pengubahan atribut menggunakan setAttribute(), serta pengelolaan gaya CSS melalui properti style. Operasi penambahan dan penghapusan node dilakukan dengan appendChild() dan removeChild(), di mana setiap perubahan langsung tercermin pada tampilan halaman tanpa memerlukan reload penuh, sehingga meningkatkan efisiensi interaksi pengguna.
Event Handling berfungsi untuk mendeteksi dan merespons interaksi pengguna seperti klik, input teks, atau scroll melalui metode addEventListener() yang diterapkan pada objek EventTarget. Proses propagasi event terjadi melalui mekanisme bubbling (dari elemen target menuju root) atau capturing (sebaliknya), dengan objek event menyediakan informasi detail seperti target, clientX, dan clientY. Konsep event delegation memungkinkan pemantauan event pada satu elemen parent untuk menangani multiple child elements, yang sangat efektif dalam mengoptimalkan performa pada daftar elemen dinamis.

D.2 Keterkaitan dengan Teknik Industri

DOM Manipulation & Event Handling digunakan untuk:

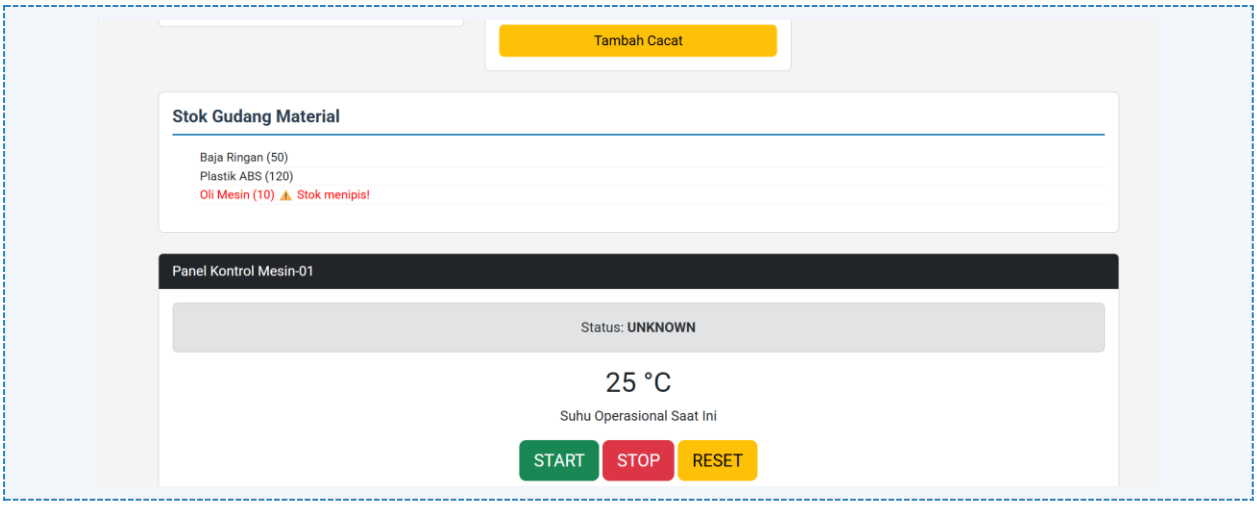
1. Dashboard monitoring produksi real-time - Tampilkan OEE, cycle time, defect rate dari sensor IoT tanpa refresh halaman
2. Simulasi facility layout - Klik mesin → hitung rectilinear distance/centroid → update visual layout instan
3. Quality control web app - Event delegation pantau multiple Six Sigma charts dari satu listener parent

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Membuat UI Panel Kontrol

```
    <!-- Simulasi Panel Kontrol Mesin (Meeting 6) -->
     <div class="card mt-4">
        <div class="card-header bg-dark text-white">Panel Kontrol Mesin-
01</div>
        <div class="card-body text-center">
            <!-- Indikator Status -->
             <div id="statusIndicator" class="alert alert-secondary"
role="alert">
                Status: <strong>UNKNOWN</strong>
            </div>

            <h2 id="suhuMesin">25 °C</h2>
            <p>Suhu Operasional Saat Ini</p>
            <div class="d-grid gap-2 d-md-block">
                <button id="btnStart" class="btn btn-success btn-
lg">START</button>
                <button id="btnStop" class="btn btn-danger btn-
lg">STOP</button>
                <button id="btnReset" class="btn btn-warning btn-
lg">RESET</button>
            </div>
        </div>
    </div>
```

Screenshot Hasil:

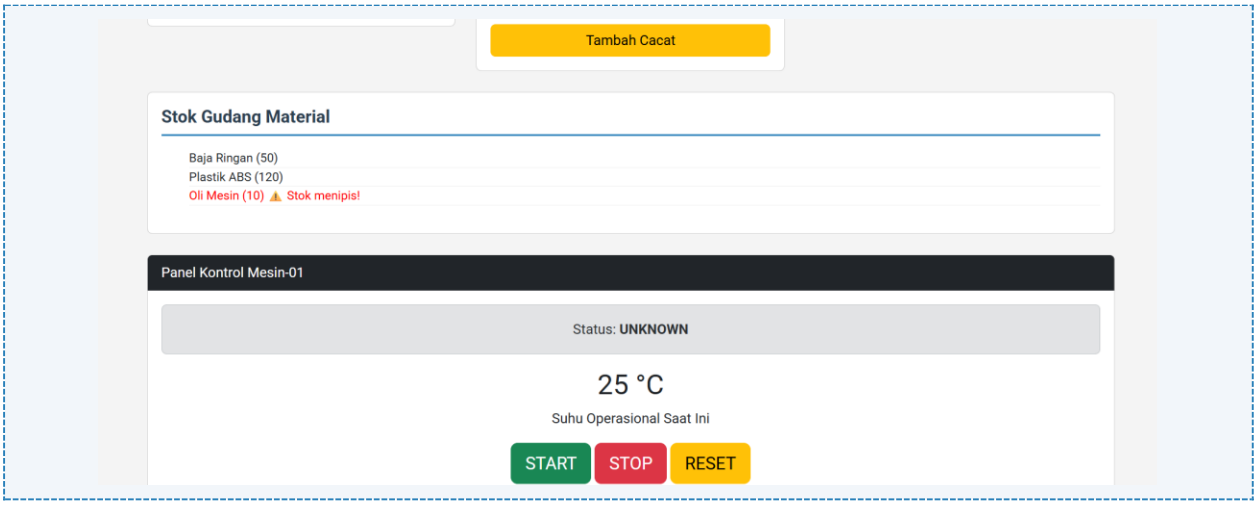


Gambar E.1 – Hasil Langkah 1

E.2 Langkah 2 – Seleksi Elemen di JavaScript

```
// 1. Seleksi Elemen DOM
const btnStart = document.getElementById('btnStart');
const btnStop = document.getElementById('btnStop');
const btnReset = document.getElementById('btnReset');
const statusIndicator = document.getElementById('statusIndicator');
const suhuMesin = document.getElementById('suhuMesin');
const teksStatus = statusIndicator.querySelector('strong'); // Mengambil
elemen <strong>
di dalam alert
// Variabel State
let suhu = 25;
let intervalId = null; // Untuk menyimpan ID timer
```

Screenshot Hasil:



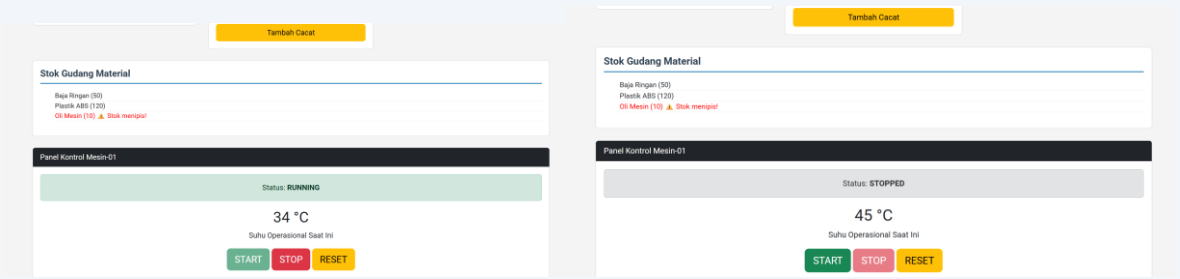
Gambar E.2 – Hasil Langkah 2

E.3 Langkah 3 – Menambahkan Event Listener (Click)

```
// Event Listener Tombol START
btnStart.addEventListener('click', function() {
 // Ubah UI Status
 statusIndicator.className = 'alert alert-success'; // Ganti class Bootstrap
(Hijau)
 teksStatus.innerText = 'RUNNING';

 // Logika simulasi kenaikan suhu
 intervalId = setInterval(function() {
 suhu += 1;
 suhuMesin.innerText = suhu + " °C";

 // Peringatan jika suhu overheat
 if (suhu > 80) {
 statusIndicator.className = 'alert alert-danger';
 teksStatus.innerText = 'OVERHEAT WARNING';
 suhuMesin.style.color = 'red';
 }
 }, 1000); // Jalankan setiap 1 detik

 // Matikan tombol Start agar tidak double click
 btnStart.disabled = true;
 btnStop.disabled = false;
});
// Event Listener Tombol STOP
btnStop.addEventListener('click', function() {
    clearInterval(intervalId); // Hentikan timer kenaikan suhu
 statusIndicator.className = 'alert alert-secondary';
 teksStatus.innerText = 'STOPPED';

 // Reset tombol
 btnStart.disabled = false;
 btnStop.disabled = true;
});
// Event Listener Tombol RESET
btnReset.addEventListener('click', function() {
 clearInterval(intervalId);
 suhu = 25;
 suhuMesin.innerText = suhu + " °C";
 suhuMesin.style.color = 'black';
 statusIndicator.className = 'alert alert-secondary';
 teksStatus.innerText = 'UNKNOWN';

 btnStart.disabled = false;
```

```
 btnStop.disabled = true;
});
```

Screenshot Hasil:



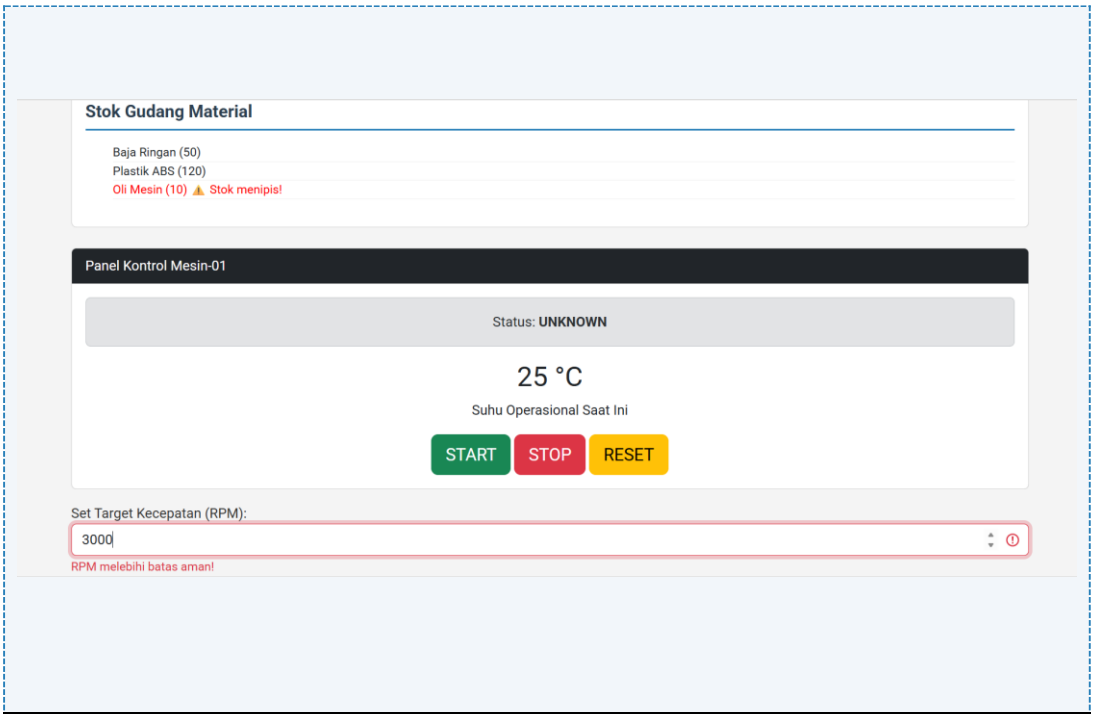
Gambar E.3 – Hasil Langkah 3

E.4 Langkah 4 – Event Input (Validasi Real-Time)

```
HTML
    <div class="mt-3">
        <label>Set Target Kecepatan (RPM):</label>
        <input type="number" id="inputRPM" class="form-control"
placeholder="Max 2000 RPM">
        <small id="pesanError" class="text-danger d-none">RPM melebihi batas
aman!</small>
    <div>

JAVASCRIPT
const inputRPM = document.getElementById('inputRPM');
const pesanError = document.getElementById('pesanError');
inputRPM.addEventListener('input', function() {
 let val = parseInt(this.value);

 if (val > 2000) {
 pesanError.classList.remove('d-none'); // Tampilkan pesan error
 this.classList.add('is-invalid'); // Berikan border merah Bootstrap
 } else {
```

```
 pesanError.classList.add('d-none'); // Sembunyikan pesan
 this.classList.remove('is-invalid');
 }
});
```

Screenshot Hasil:



Gambar E.4 – Hasil Langkah 4

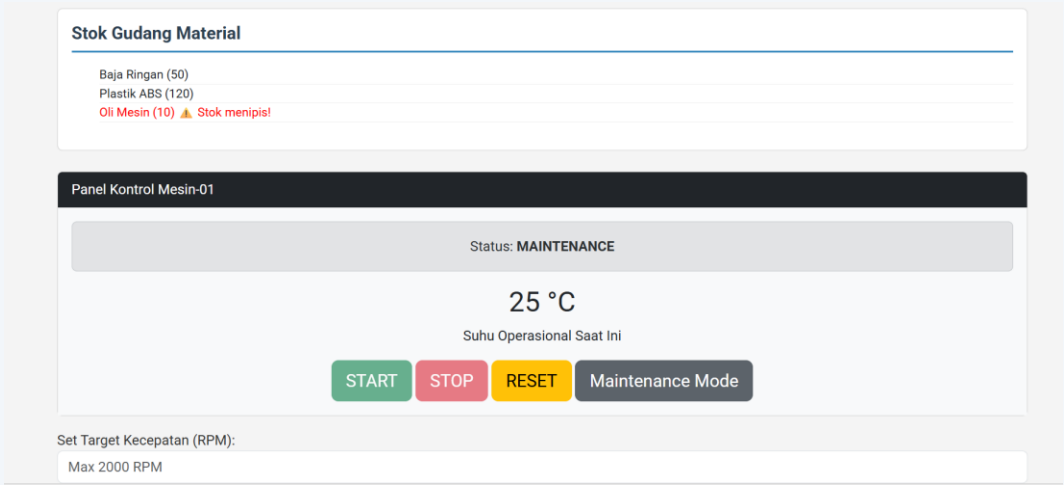
F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – DOM Styling

Buatlah tombol tambahan bernama "Maintenance Mode". Saat diklik, background dari card menjadi abu-abu (`bg-light`), dan teks status berubah menjadi "MAINTENANCE".

Jawaban / Kode Solusi:

```
HTML
<button class="btn btn-secondary btn-lg"
onclick="maintenanceMode()">Maintenance Mode</button>

JAVASCRIPT
// LATIHAN 1
function maintenanceMode() {
    clearInterval(intervalId);

    // Ubah teks status
    teksStatus.innerText = "MAINTENANCE";

    // Ubah warna alert indicator
    statusIndicator.className = "alert alert-secondary";

    // Ambil card Panel Kontrol secara spesifik lewat statusIndicator
    let cardBody = statusIndicator.closest('.card-body');
    cardBody.classList.add('bg-light');

    // Reset tombol
    btnStart.disabled = true;
    btnStop.disabled = true;
}
```



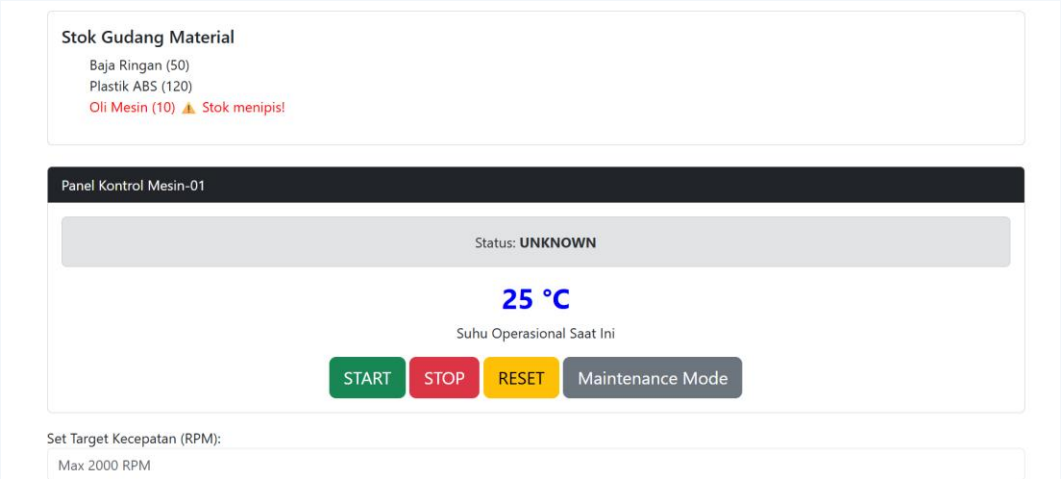
Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 – Mouse Events

Tambahkan efek saat mouse diarahkan (`mouseover`) ke teks suhu mesin, suhu tersebut menjadi tebal dan berwarna biru. Saat mouse keluar (`mouseout`), kembali normal.

Jawaban / Kode Solusi:

```
// LATIHAN 2 - Mouseover Suhu Mesin
suhuMesin.addEventListener('mouseover', function() {
    suhuMesin.style.fontWeight = 'bold';
    suhuMesin.style.color = 'blue';
});

suhuMesin.addEventListener('mouseout', function() {
    suhuMesin.style.fontWeight = 'normal';
    suhuMesin.style.color = 'black';
});
```



Gambar F.2 – Hasil Latihan 2

F.3 Tugas Proyek Mini – Kalkulator Tagihan Listrik

Deskripsi proyek mini:

a. Buat UI dengan 2 input: "Daya (Watt)" dan "Jam Pemakaian".
b. Saat user mengetik di salah satu input, secara real-time (tanpa tombol submit) tampilkan hasil "Total kWh" dan "Estimasi Biaya (Rp 1.500/kWh)" di elemen `di bawahnya.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini Kalkulator Tagihan Listrik, pendekatan yang dilakukan dimulai dari seleksi elemen DOM menggunakan document.getElementById() untuk mengambil elemen input Daya, input Jam, dan elemen hasil agar bisa dimanipulasi melalui JavaScript. Setelah elemen berhasil diambil, ditambahkan event listener dengan event input pada kedua input tersebut sehingga setiap kali user mengetik, function hitungListrik() langsung dipanggil secara real-time tanpa perlu menekan tombol apapun.

Di dalam function hitungListrik(), ditambahkan validasi input menggunakan isNaN() dan pengecekan nilai negatif untuk memastikan perhitungan hanya dilakukan jika input valid. Jika input tidak valid, elemen hasil disembunyikan kembali menggunakan classList.add('d-none'). Jika valid, dilakukan perhitungan konversi Watt ke kWh dengan rumus (daya * jam) / 1000 kemudian dikalikan tarif Rp 1.500 untuk mendapatkan estimasi biaya.

Kode Utama:

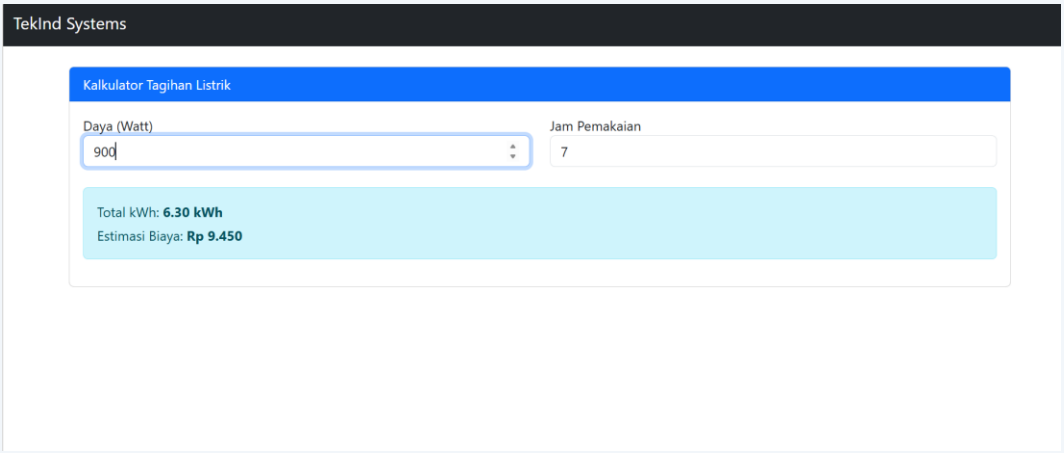
HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Perhitungan Lingkaran</title>

    <!-- Bootstrap -->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">

    <!-- CSS tambahan -->
    <link rel="stylesheet" href="latihan1.css">
</head>

<body>

<!-- NAVBAR -->
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container-fluid">
        <a class="navbar-brand" href="">TekInd Systems</a>
    </div>
</nav>
<div class="container mt-4">
<!-- PROYEK MINI - Kalkulator Tagihan Listrik -->
<div class="card mt-4">
    <div class="card-header bg-primary text-white">
        Kalkulator Tagihan Listrik
    </div>
    <div class="card-body">
        <div class="row">
            <div class="col-md-6">
                <label>Daya (Watt)</label>
                <input type="number" id="inputDaya" class="form-control mb-2"
placeholder="Masukkan daya (Watt)">
            </div>
            <div class="col-md-6">
                <label>Jam Pemakaian</label>
                <input type="number" id="inputJam" class="form-control mb-2"
placeholder="Masukkan jam pemakaian">
            </div>
        </div>
```

```
        <div id="hasilListrik" class="alert alert-info mt-3 d-none">
            <p class="mb-1">Total kWh: <strong id="totalKwh"></strong></p>
            <p class="mb-0">Estimasi Biaya: <strong
id="totalBiaya"></strong></p>
        </div>
    </div>
</div>
</div>

<script src="Kalkulator Tagihan Listrik.js"></script>

</body>
</html>
```

JAVASCRIPT

```
// PROYEK MINI - Kalkulator Tagihan Listrik
const inputDaya = document.getElementById('inputDaya');
const inputJam = document.getElementById('inputJam');
const hasilListrik = document.getElementById('hasilListrik');
const totalKwh = document.getElementById('totalKwh');
const totalBiaya = document.getElementById('totalBiaya');

function hitungListrik() {
    let daya = parseFloat(inputDaya.value);
    let jam = parseFloat(inputJam.value);

    if (isNaN(daya) || isNaN(jam) || daya <= 0 || jam <= 0) {
        hasilListrik.classList.add('d-none');
        return;
    }

    let kwh = (daya * jam) / 1000;
    let biaya = kwh * 1500;

    totalKwh.innerText = kwh.toFixed(2) + " kWh";
    totalBiaya.innerText = "Rp " + biaya.toLocaleString('id-ID');

    hasilListrik.classList.remove('d-none');
}

inputDaya.addEventListener('input', hitungListrik);
inputJam.addEventListener('input', hitungListrik);
```



Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Pertama, dari sisi teori DOM Manipulation, kode yang dibuat sudah sesuai karena menggunakan document.getElementById() untuk mengambil elemen HTML dan memanipulasinya melalui JavaScript. Hal ini sesuai dengan konsep DOM (Document Object Model) dimana JavaScript berinteraksi dengan elemen HTML melalui seleksi elemen terlebih dahulu sebelum melakukan perubahan.

Kedua, dari sisi teori Event Handling, penggunaan addEventListener('input', hitungListrik) sudah sesuai dengan konsep event-driven programming dimana program merespons aksi yang dilakukan user. Event input dipilih karena bersifat real-time sehingga function langsung dipanggil setiap kali user mengetik tanpa menunggu tombol ditekan, berbeda dengan event click yang hanya merespons saat tombol diklik.

Ketiga, dari sisi teori Function, pembuatan hitungListrik() sudah sesuai dengan konsep function declaration dimana logika perhitungan dikelompokkan dalam satu function yang dapat dipanggil dari berbagai tempat, baik dari event listener maupun dari tombol melalui onclick.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Kendala yang sering ditemukan adalah lupa menambahkan return berupa object yang berisi dua nilai sekaligus yaitu luas dan keliling, karena biasanya function hanya mengembalikan satu nilai saja.	Mengembalikan dua nilai sekaligus dengan membungkusnya dalam sebuah object menggunakan kurung kurawal
Tambah Job bisa diklik berkali-kali sehingga job J04 masuk ke array lebih dari satu kali.	Menonaktifkan tombol setelah job baru berhasil ditambahkan menggunakan disabled, sehingga tombol hanya bisa diklik satu kali saja.
Tombol Tambah Cacat tidak bisa diklik karena function tambahData() belum didefinisikan di JS.	Mendefinisikan function tambahData() dan renderListCacat() di JS. renderListCacat() bertugas mengosongkan list lama lalu me-render ulang seluruh isi array dari awal, sehingga setiap kali ada penambahan data tampilan selalu sinkron dengan isi array.

G.3 Kaitan dengan Penerapan di Industri

Ketiga solusi yang dipakai dalam praktikum ini sebetulnya mencerminkan hal-hal yang memang terjadi di dunia industri nyata. Pertama, soal return object di function hitungLingkaran(). dalam pekerjaan engineer di industri, jarang sekali satu perhitungan hanya menghasilkan satu angka saja. Misalnya waktu menghitung performa mesin, hasilnya bisa sekaligus berupa efisiensi energi dan efisiensi waktu, jadi masuk akal kalau satu function langsung mengembalikan dua nilai sekaligus daripada hitung dua kali. Kedua, soal tombol Tambah Job yang dibatasi supaya tidak bisa diklik berkali-kali — ini sebenarnya mirip dengan aturan di lantai produksi dimana satu pekerjaan tidak boleh dijadwalkan dua kali ke mesin yang sama, karena kalau sampai terjadi maka jadwal produksi bisa kacau dan mesin terpakai sia-sia. Ketiga, soal function pencatatan cacat yang langsung

memperbarui tampilan setiap ada data baru — di pabrik sungguhan, data cacat tidak boleh menunggu sampai akhir shift baru dilaporkan karena supervisor butuh tahu kondisi kualitas produksi saat itu juga supaya bisa langsung ambil keputusan. Jadi ketiga solusi koding ini bukan sekadar menyelesaikan masalah teknis, tapi juga mencerminkan cara berpikir yang memang dibutuhkan dalam pengelolaan produksi dan kualitas di industri.

H. KESIMPULAN

1	Kode praktikum berhasil menerapkan konsep DOM Manipulation (getElementById()), Event Handling (addEventListener('input')), dan Function Declaration (hitungListrik()) sesuai prinsip event-driven programming untuk interaksi real-time.
2	Masalah return multiple value diatasi dengan object return, duplikasi data dicegah dengan tombol disabled, dan fungsi undefined diperbaiki dengan definisi tambahData() serta renderListCacat() untuk sinkronisasi tampilan-array.
3	

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
1.	Galahartlambang, Y., Khotiah, T., & Jumain. (2021). Analisa Performa Aplikasi Web Berbasis Manipulasi DOM dan Virtual DOM. Seminar Nasional Inovasi Teknologi (INOTEK), 36-40.
2.	Rosidi, H. (2025). Sistem Antrian Cucian Mobil Berbasis Website Menggunakan Konsep CRM dan Penerapan UML [Skripsi]. Universitas Bina Sarana Informatika.
3.	

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
Saprina Melita Sari NIM: 23051430012	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____